Exercise A: User Manual Procedure

How to Create and Activate a Python Virtual Environment and Install a Package (Windows)

Overview

When you build Python projects, each project may need different tools. A **virtual environment** is a private toolbox for one project, so the tools of one project never mix with another. In this guide you will create a folder, build a toolbox inside it, switch the toolbox on, and put one tool inside it.

Prerequisites

This guide is for first-semester computing students who can use a computer but have never used the Command Prompt or Python tools. It covers Windows 10 and 11.

You need:

- A computer running Windows 10 or 11
- Python 3.8 or newer installed (Step 12 shows you how to check)
- An internet connection
- About 15 minutes

You do **not** need to know how to use the Command Prompt or how to create folders with code. This guide shows every action.

Key Terms

- **Command Prompt:** a window where you control the computer by typing instructions instead of clicking.
- **Command:** one instruction that you type into the Command Prompt.
- **Folder:** a container on your computer that holds files.
- **Virtual environment:** a private toolbox for one project, with its own set of Python tools.
- **Package:** a ready-made piece of code written by someone else that you can add to your project.
- **pip:** the tool that downloads and installs packages.

How to Read the Steps

- Words in `this style` are typed exactly as shown.
- In this guide, **run** means: type the command, then press the **Enter** key.

- Each step ends with **Result:** so you can check that you did it correctly before you continue.
- If your result does not match, stop and do not continue to the next step.

Part 1: Create Your Project Folder

Step 1. Click the yellow folder icon on the taskbar (the bar at the bottom of your screen). **Result:** A window called File Explorer opens.

Step 2. Click **Documents** in the list on the left side of the window. **Result:** The window shows what is inside your Documents folder. It may be empty.

Step 3. Right-click on an empty white space in the window. **Result:** A small menu appears next to your mouse.

Step 4. Move your mouse over **New** in the menu. **Result:** A second menu slides out to the side.

Step 5. Click **Folder** in the second menu. **Result:** A new folder appears with the name "New folder". The name is highlighted in blue, which means you can type a new name.

Step 6. Type `myproject`. **Result:** The blue name changes to `myproject`.

Step 7. Press the **Enter** key. **Result:** The folder is now named `myproject`.

Part 2: Open the Command Prompt

Step 8. Hold the **Windows key** (the key with the four-square logo, near the bottom-left of your keyboard) and press **R**. **Result:** A small box called **Run** appears at the bottom-left of your screen.

Step 9. Type `cmd` in the Run box. **Result:** The letters `cmd` appear in the box.

> **Screenshot 1 (place after Step 9) What it shows:** The Run box at the bottom-left of the screen with `cmd` typed in the "Open:" field. **What to highlight:** A red rectangle around the "Open:" field and an arrow pointing to the **OK** button. **Caption:** "The Run box with `cmd` typed in." **Why it helps:** The reader can confirm they opened the right box and typed in the right place.

Step 10. Press the **Enter** key. **Result:** A black window opens. This is the Command Prompt. The last line ends with a `>` symbol and a blinking cursor, like this: `C:\Users\Wanjiru>`. Your name will appear instead of "Wanjiru".

On Windows 11, the window may have a tab at the top. This is normal. Continue with Step 11.

Part 3: Go to Your Project Folder

Step 11. Run `cd Documents\myproject`. (`cd` stands for "change directory". It tells the Command Prompt to go inside a folder, like double-clicking it.) **Result:** The last line changes to end with `Documents\myproject>`. The Command Prompt is now inside your folder.

Part 4: Check Python

Step 12. Run `python --version`. **Result:** The Command Prompt shows a version number, for example `Python 3.12.4`. Your number may be different. Any number that starts with 3.8 or higher is fine.

If you see an error instead, go to **Troubleshooting** at the end of this guide before continuing.

Part 5: Create and Activate the Virtual Environment

Step 13. Run `python -m venv venv`. (This builds your private toolbox and names it `venv`. The command may need a few seconds.) **Result:** No message appears. After a few seconds, the cursor returns on a new line ending in `myproject>`. A new folder named `venv` now exists inside `myproject`. You can look in File Explorer to confirm.

Step 14. Run `venv\Scripts\activate`. (This switches your toolbox on.) **Result:** The text `(venv)` appears at the very start of the last line, like this: `(venv) C:\Users\Wanjiru\Documents\myproject>`.

> **Screenshot 2 (place after Step 14) What it shows:** The black Command Prompt window. The last line begins with `(venv)` in front of the folder path. **What to highlight:** A red rectangle around `(venv)` with the label "This means your virtual environment is ON." **Caption:** "The Command Prompt after activation. `(venv)` appears at the start of the line." **Why it helps:** This is the main sign that activation worked. Readers can compare their own screen with this image.

Part 6: Install a Package

Step 15. Run `pip install requests`. (`requests` is a popular package that lets Python talk to websites. You need an internet connection for this step.) **Result:** Text scrolls down the window for a few seconds. The last line begins with `Successfully installed` and includes the word `requests`.

Step 16. Run `pip show requests`. **Result:** The window shows a list of details. The first line is `Name: requests` and the second line shows a version number. This confirms that the package is installed in your virtual environment.

You are done. You created a folder, built a virtual environment, switched it on, and installed a package inside it.

When you finish working, run `deactivate`. The `(venv)` text disappears, which means the environment is switched off. To use it again, open the Command Prompt (Steps 8 to 10), then repeat Step 11 and Step 14.

Troubleshooting: `'python' is not recognized`

What you see in Step 12:

'python' is not recognized as an internal or external command,
operable program or batch file.

What it means: Either Python is not installed, or it was installed without telling Windows where to find it. This is the most common beginner error.

How to fix it:

1. Close the Command Prompt.
2. Go to **python.org/downloads** and download the latest Python for Windows.
3. Open the downloaded file. On the **first** screen, tick the box **Add python.exe to PATH**, then click **Install Now**.
4. When it finishes, open a **new** Command Prompt (Steps 8 to 10) and repeat from Step 11.

Result: `python --version` now shows a version number.

Exercise B: API Reference Entry

Create a Task

Endpoint
POST /api/v1/projects/{project_id}/tasks

Description

This endpoint creates a new task inside a project. You send the task's details (title, assignee, due date, and priority) and the server saves the task and returns it with a new unique ID.

Only a logged-in user who belongs to the project can create tasks in it. The person you assign the task to must also belong to the project.

Authentication

Every request must include an access token in the `Authorization` header. You get a token when you log in. A request without a valid token is rejected with `401 Unauthorized`.

Request Headers

Header	Value	Required	Description
`Authorization`	`Bearer <token>`	Yes	Your access token. Replace `<token>` with the token you received at login.
`Content-Type`	`application/json`	Yes	Tells the server that the request body is JSON.
`Accept`	`application/json`	No	Asks for the response in JSON. JSON is the default, so you can leave this out.

Path Parameters

Name	Type	Required	Description
`project_id`	integer	Yes	The unique ID of the project that will contain the new task. Example: `305`.

Query Parameters

Name	Type	Required	Default	Description
`notify`	boolean	No	`false`	If `true`, the server sends an email to the assignee when the task is created. Allowed values: `true` or `false`.

Request Body Parameters

The body must be a JSON object with these fields.

Name	Type	Required	Description
`title`	string	Yes	A short name for the task. Length: 1 to 100 characters.
`description`	string	No	More details about the task. Maximum 1,000 characters. If you leave it out, the response shows `null`.
`assignee_id`	integer	Yes	The user ID of the person responsible for the task. This person must be a member of the project.
`due_date`	string	Yes	The date the task must be finished, in `YYYY-MM-DD` format. It must be today or a future date. Example: `2026-11-15`.
`priority`	string	Yes	How urgent the task is. Allowed values: `low`, `medium`, or `high` (lowercase only).

Example Request

Request body:

```
{
  "title": "Design the landing page for the school fees portal",
  "description": "Create a mobile-friendly layout. Parents should be able to pay by M-Pesa.",
  "assignee_id": 17,
  "due_date": "2026-11-15",
  "priority": "high"
}
```

Full request using curl (curl is a command-line tool for sending requests):

```
curl -X POST "https://api.example.com/api/v1/projects/305/tasks?notify=true" \
  -H "Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.example" \
  -H "Content-Type: application/json" \
  -d '{
```

```
  "title": "Design the landing page for the school fees portal",
  "description": "Create a mobile-friendly layout. Parents should be able to pay by M-Pesa.",
  "assignee_id": 17,
  "due_date": "2026-11-15",
  "priority": "high"
}'
```

Response Codes

Code	Name	When it happens
`201`	Created	The task was created successfully. The response body contains the new task.
`400`	Bad Request	The request body is not valid JSON. For example, a comma or a bracket is missing.
`401`	Unauthorized	The `Authorization` header is missing, or the token is invalid or expired.
`403`	Forbidden	The token is valid, but you do not belong to this project, so you cannot create tasks in it.
`404`	Not Found	No project exists with the `project_id` you sent.
`415`	Unsupported Media Type	The `Content-Type` header is missing or is not `application/json`.
`422`	Unprocessable Entity	The JSON is valid, but the values are wrong. Examples: a required field is missing, `priority` is not `low`, `medium` or `high`, `due_date` is in the past or in the wrong format, `title` is too

		long, or `assignee_id` does not belong to a member of the project. The response lists each problem.
`429`	Too Many Requests	You sent too many requests in a short time (limit: 60 per minute). Wait a few seconds and try again.
`500`	Internal Server Error	Something went wrong on the server. This is not your mistake. Try again later, and contact support if it continues.
`503`	Service Unavailable	The server is temporarily down, for example during maintenance. Try again later.

Example Successful Response

Status: `201 Created`


```
{
  "id": 1042,
  "project_id": 305,
  "title": "Design the landing page for the school fees portal",
  "description": "Create a mobile-friendly layout. Parents should be able to pay by M-Pesa.",
  "assignee_id": 17,
  "due_date": "2026-11-15",
  "priority": "high",
  "status": "to_do",
  "created_by": 8,
  "created_at": "2026-10-23T08:45:12Z"
}
```


Response Body Fields

Name	Type	Description
`id`	integer	The unique ID of the new task. Use it to view, edit, or delete the task later.

`project_id`	integer	The ID of the project that contains the task.
`title`	string	The task title you sent.
`description`	string or null	The description you sent, or `null` if you left it out.
`assignee_id`	integer	The user ID of the assignee.
`due_date`	string	The due date in `YYYY-MM-DD` format.
`priority`	string	`low`, `medium`, or `high`.
`status`	string	The task's progress. Every new task starts as `to_do`.
`created_by`	integer	The user ID of the person who created the task (taken from your token).
`created_at`	string	The date and time the task was created, in UTC (ISO 8601 format).

Example Error Response

Status: `422 Unprocessable Entity`


```
{
  "error": "validation_failed",
```

```
  "message": "The request contains invalid values.",
  "details": [
    {
      "field": "priority",
      "issue": "Must be one of: low, medium, high."
    },
    {
      "field": "due_date",
      "issue": "Must be today or a future date."
    }
  ]
}
```